

Accountable Credit Vaults Audit Report

Version 1.1

Conducted by:
Alin Barbatei (ABA) — Lead
Silverologist

April 25, 2025

Table of Contents

1	Introduction	4
1.1	About ABA and Silverologist	4
1.2	About Accountable Credit Vaults	4
1.3	Issues Risk Classification	5
	Impact	5
	Likelihood	5
	Actions required by severity level	5
	Informational findings	5
2	Executive Summary	6
2.1	Overview	6
2.2	Audit Scope	6
2.3	Summary of Findings	6
2.4	Findings & Resolutions	7
3	Findings	9
3.1	Critical Severity Findings	9
	[C-01] Incorrect share-to-asset comparison enables asset theft	9
3.2	High Severity Findings	10
	[H-01] Canceling a request blocks new deposit and redeem requests	10
	[H-02] Incorrect share price used when claiming requests	11
	[H-03] Scaling conversions removes rounding-up leading to broken accounting	13
	[H-04] Unauthorized redeem requests allow forced redemptions and griefing	15
3.3	Medium Severity Findings	16
	[M-01] Accountable vaults are not compliant with ERC-4626, ERC-7540 and ERC-7575	16
	[M-02] Frontrunning fulfillment transactions leads to vault depletion	17
	[M-03] Restrictive whitelisted-lender permission model introduces economical constraint	18
3.4	Low Severity Findings	20
	[L-01] Protocol does not support asset decimals greater than share price decimals	20
	[L-02] Incorrectly fulfilling requests breaks accounting	20
	[L-03] Possible event spamming due to missing 0 input amount checks	22
	[L-04] Vault claim events can be spammed	22
3.5	Informational Findings	24
	[I-01] Transfer whitelists can't be added	24
	[I-02] Add indexing for IAccountableLoan events	24
	[I-03] Rounding discrepancy between fulfilling and claiming can cause inconveniences	24
	[I-04] Missing event emission for important actions	25
	[I-05] Soulbound vault shares can be transferred by abusing the redeemRequest and cancelRedeem mechanism	26
	[I-06] Missing input sanitization on important setters	27
	[I-07] _totalPendingRedeemAssets is not used	27
	[I-08] Remove debug remnants from vaults	28

[I-09] Add security contact to contracts	28
4 System design notes	29
4.1 ERC-7540 integration	29
4.2 Loan managers	29
4.3 Tranche tokens	31
4.4 General code advice	32
Eliminate subtractions when possible	32
Vault upgradability	32
Fulfillment validations	32
5 Disclaimer	33

1 Introduction

1.1 About ABA and Silverologist

ABA, or Alin Mihai Barbatei, is an established independent security researcher with deep expertise in blockchain security. With a background in traditional information security and competitive hacking, ABA has a proven track record of discovering hidden vulnerabilities. He has extensive experience in securing both EVM (Ethereum Virtual Machine) compatible blockchain projects and Bitcoin L2, Stacks projects.

Having conducted several solo and collaborative smart contract security reviews, ABA consistently strives to provide top-quality security auditing services. His dedication to the field is evident in the top-notch, high-quality, comprehensive smart contract auditing services he offers.

To learn more about his services, visit ABA's website abarbatei.xyz. You can also view his [audit portfolio here](#). For audit bookings and security review inquiries, you can reach out to ABA on Telegram, Twitter (X) or WarpCast:

🔗 <https://t.me/abarbatei>

✉ <https://x.com/abarbatei>

📄 <https://warpcast.com/abarbatei.eth>

Silverologist ([@silverologist](#)), a promising junior auditor with remarkable results in competitive audits, contributed his expertise to help secure the protocol.

1.2 About Accountable Credit Vaults

The [Accountable](#) Credit Vaults system is designed to allow uncollateralized borrowing and lending. This is achieved through an [ERC7540](#) compatible vault system where lenders deposit market tokens and trusted borrowers tap into those funds via a loan manager contract.

The protocol and design is still in the development phase and will suffer further changes. The end goal is providing a base for any RWA (Real World Assets) on-chain loans.

1.3 Issues Risk Classification

The current report contains issues, or findings, that impact the protocol. Depending on the likelihood of the issue appearing and its impact (damage), an issue is in one of four risk categories or severities: *Critical, High, Medium, Low* or *Informational*.

The following table show an overview of how likelihood and impact determines the severity of an issue.

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost or a functionality of the protocol is affected.
- **Low** - any kind of unexpected behavior that's not so critical.

Likelihood

- **High** - direct attack vector; the cost is relatively low to the amount of funds that can be lost.
- **Medium** - only conditionally incentivized attack vector, but still relatively likely.
- **Low** - too many or too unlikely assumptions; provides little or no incentive.

Actions required by severity level

- **Critical** - client **must** fix the issue.
- **High** - client **must** fix the issue.
- **Medium** - client **should** fix the issue.
- **Low** - client **could** fix the issue.

Informational findings

Informational findings encompass recommendations to enhance code style, operations alignment with industry best practices, gas optimizations, adherence to documentation, standards, and overall contract design.

Informational findings typically have minimal impact on code functionality or security risk.

Evaluating all vulnerability types, including informational ones, is crucial for a comprehensive security audit to ensure robustness and reliability.

2 Executive Summary

2.1 Overview

Project Name	Accountable Credit Vaults
Codebase	https://github.com/Accountable-Protocol/credit
Operating platform	Monad, Ethereum, Arbitrum, Base, Optimism, Avalanche C-Chain
Programming language	Solidity
Initial commit	dd13abaa71d58f624b50ccf62f516fc8a5812e16
Remediation commit	cddf102491d866287ca624113c9e024024117673
Timeline	From 19.02.2025 to 05.03.2025 (15 days)
Audit methodology	Static analysis and manual review

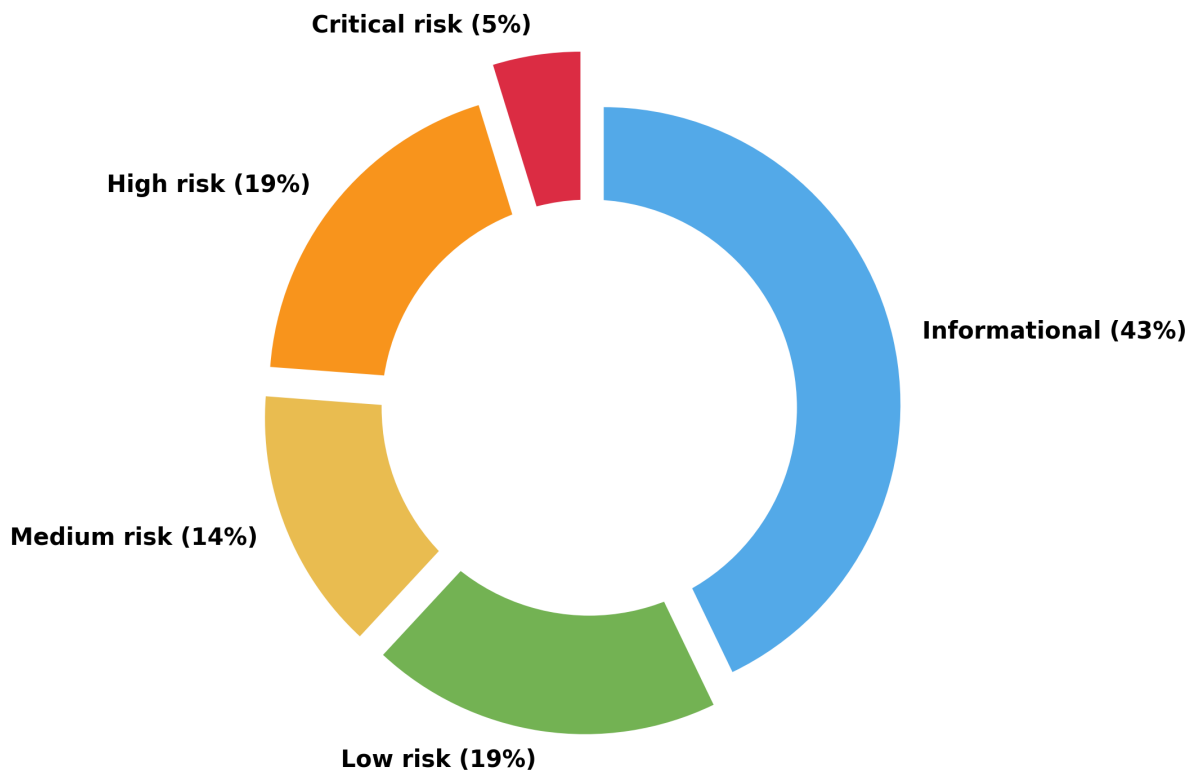
2.2 Audit Scope

Files and folders in scope

- src/loan/AccountableLoan.sol
 - src/vault/AccountableAsyncVault.sol
 - src/vault/AccountableVault.sol
-

2.3 Summary of Findings

Severity	Total Found	Resolved	Partially Resolved	Acknowledged
Critical risk	1	1	0	0
High risk	4	4	0	0
Medium risk	3	2	1	0
Low risk	4	4	0	0
Informational	9	6	0	3



2.4 Findings & Resolutions

ID	Title	Severity	Status
C-01	Incorrect share-to-asset comparison enables asset theft	Critical	Resolved
H-01	Canceling a request blocks new deposit and redeem requests	High	Resolved
H-02	Incorrect share price used when claiming requests	High	Resolved
H-03	Scaling conversions removes rounding-up leading to broken accounting	High	Resolved
H-04	Unauthorized redeem requests allow forced redemptions and griefing	High	Resolved
M-01	Accountable vaults are not compliant with ERC-4626, ERC-7540 and ERC-7575	Medium	Partially Resolved
M-02	Frontrunning fulfillment transactions leads to vault depletion	Medium	Resolved

ID	Title	Severity	Status
M-03	Restrictive whitelisted-lender permission model introduces economical constraint	Medium	Resolved
L-01	Protocol does not support asset decimals greater than share price decimals	Low	Resolved
L-02	Incorrectly fulfilling requests breaks accounting	Low	Resolved
L-03	Possible event spamming due to missing 0 input amount checks	Low	Resolved
L-04	Vault claim events can be spammed	Low	Resolved
I-01	Transfer whitelists can't be added	Informational	Resolved
I-02	Add indexing for IAccountableLoan events	Informational	Resolved
I-03	Rounding discrepancy between fulfilling and claiming can cause inconveniences	Informational	Acknowledged
I-04	Missing event emission for important actions	Informational	Resolved
I-05	Soulbound vault shares can be transferred by abusing the redeemRequest and cancelRedeem mechanism	Informational	Acknowledged
I-06	Missing input sanitization on important setters	Informational	Resolved
I-07	_totalPendingRedeemAssets is not used	Informational	Acknowledged
I-08	Remove debug remnants from vaults	Informational	Resolved
I-09	Add security contact to contracts	Informational	Resolved

3 Findings

3.1 Critical Severity Findings

[C-01] Incorrect share-to-asset comparison enables asset theft

Severity: *Critical risk* (Resolved) [PoC]

Context: *AccountableLoan.sol:198*

Description

In the `AccountableAsyncVault` contract, users can redeem their shares for assets by submitting a redemption request. The loan operator then fulfills the request and updates `state.maxRedeem`, which defines the maximum amount of **assets** the user can claim. Users can then claim their assets through either of the following functions:

- `redeem`, specifying the number of shares they want to exchange.
- `withdraw`, specifying the number of assets they want to claim.

However, the `withdraw` function fails to work correctly unless the share-to-asset ratio is exactly 1.

The issue arises from the following condition in the `withdraw` function:

```
shares = _convertToShares(vault, assets, state.redeemPrice, Math.Rounding.Floor);  
if (shares > state.maxWithdraw) revert ExceedsRedeemLimit();
```

Here, `state.maxWithdraw` is meant to represent a limit on assets, but the function incorrectly compares it against shares.

When the share-to-asset ratio is greater than 1, users can only successfully withdraw assets if their requested amount, when converted in shares, is within the `state.maxWithdraw` threshold. This means that users cannot withdraw their full claimable assets in a single transaction. Instead, they would need to perform multiple withdrawals, potentially requiring an unrealistic number of transactions.

When the share-to-asset ratio is less than 1, an attacker can successfully extract from the vault more assets than they are entitled to. Since the total pending redeem assets are tracked in `_totalPendingRedeemAssets`, this also means the legitimate owners of those assets will be blocked from withdrawing them.

Recommendation

Modify the `AccountableLoan::withdraw` function to compare the user's requested asset amount against `state.maxWithdraw`, instead of using the converted share value.

Resolution: Resolved, the recommended fix was implemented in [#640f2e1](#).

3.2 High Severity Findings

[H-01] Canceling a request blocks new deposit and redeem requests

Severity: *High risk* (Resolved) [PoC]

Context: *AccountableLoan.sol:296,314*

Description

After users cancel a pending request in the `AccountableAsyncVault`, they will no longer be able to submit another request of the same type due to an issue in the implementation.

When cancelling a redeem request, in `AccountableLoan::cancelRedeemRequest` the `pendingCancelRedeemRequest` flag is set to true, but it is never reset to false after the cancellation is fulfilled via the `fulfillCancelDepositRequest` function.

This happens because instead of setting the `pendingCancelRedeemRequest` to false, the `pendingDepositRequest` state variable is incorrectly removed instead.

```
if (state.pendingDepositRequest == 0) delete state.pendingDepositRequest;
```

This pending flag then *blocks any other request* from being submitted.

The same issue exists in `AccountableLoan::fulfillCancelRedeemRequest`, where `pendingRedeemRequest` is not reset when the `pendingRedeemRequest` reaches 0.

```
if (state.pendingRedeemRequest == 0) delete state.pendingRedeemRequest;
```

Users will be blocked from making new deposit or redeem requests after canceling their initial request (without utilizing the operator delegating mechanism).

Recommendation

After a cancel request is fulfilled, reset the pending cancel request flag in each case.

In the `_fulfillCancelDepositRequest` function, change the incorrect delete statement:

```
if (state.pendingDepositRequest == 0) delete state.pendingDepositRequest;
```

to actually set `state.pendingCancelRedeemRequest` to false.

Likewise, in the `_fulfillCancelRedeemRequest` function, set its corresponding `pendingCancelRedeemRequest` flag to false when `state.pendingRedeemRequest == 0`.

Resolution: Resolved, the recommended fix was implemented in [#640f2e1](#).

[H-02] Incorrect share price used when claiming requests

Severity: *High risk* (Resolved) [PoC]

Context: [AccountableLoan.sol:254,275](#)

Description

When a deposit or redeem request is fulfilled by the loan operator in `AccountableLoan`, the fulfillment price is stored in `state.depositPrice` and `state.redeemPrice`, respectively, and the available assets or share to `deposit` or `redeem` are calculated and stored at that point in time.

This price is later used during the claim process to determine the `asset` or `share` amounts for request claiming.

However, if a user submits multiple requests of the same type and waits for all to be fulfilled before claiming them, the last request fulfillment price will be used for all requested amounts, leading to a mismatch between the noted available amounts and actually used amounts. In some cases it also leads to reverts and reverts.

This happens because the price is stored per vault and owner, rather than per individual request for each owner.

```
function _fulfillDepositRequest(address vault, address owner, uint256 assets)
    internal virtual {
    @> VaultState storage state =
        _getAccountableLoanStorage()._vaultStates[vault][owner];
        if (state.pendingDepositRequest == 0) revert NoDepositRequest();

    @> uint256 price = _sharePrice(vault);
        uint256 shares = _convertToShares(vault, assets, price,
            Math.Rounding.Floor);

    @> state.depositPrice = price;
```

and then that latest price is used for claiming the operation.

Recommendation

Contrast to ERC4626 vaults where operations are done atomically, with async vaults ERC7540, multiple requests may be submitted in parallel and claimed all at one. Due to the nature of this design, a universally accepted solution that provides fair price association when claiming is not realistically achievable.

However, several solutions may be applied, with varying degrees of success by simulating how shares are priced when selling stocks for tax purposes.

The following pricing implementations can be used as a fix to the issue:

1. FIFO (First-In, First-Out)

For each request (either redeem/withdraw or deposit) note the exact shares and assets resulting pair in a list. When claiming, iterate through requests in order of the first to be requested to the last, and consume the noted available assets and shares

2. LIFO (Last-In, First-Out)

Similar to FIFO works, but the order of iterating through the noted requests is backwards, from the last entry to the first

3. Average Cost Basis

On each request, have an accumulator for resulted shares and one for assets. Calculate the average exchange rate (cost) by dividing the total amount in assets by the number of shares. A lender who has requested deposit 10,000 USDC in the vault which would be equivalent to 500 shares would have an average cost basis of 20 USDC, which will be the rate used at claiming.

Out of the mentioned solutions, an average cost basis implementation seems more suited for the current vault system. Also, using the FIFO/LIFO system may add overhead and gas issues if the pending lists overgrows (e.g. a compromised operator initiates multiple dust requests to DOS the original lender).

Resolution: Resolved. A fix was implemented in [#640f2e1](#) and [#cddf102](#).

Accountable: resolved using average cost basis.

[H-03] Scaling conversions removes rounding-up leading to broken accounting

Severity: *High risk* (Resolved) [PoC]

Context: [AccountableLoan.sol:332-365](#)

Description

The `AccountableLoan` contract is responsible for determining how assets are converted to shares and back.

The current implementation, found in `_convertToAssets` and `_convertToShares`, incorrectly removes rounding-up, resulting in abusive accounting issues.

Both conversion functions first scale up the resource (assets or shares) amount to the vault share price decimals, then do the conversion with rounding. After this, the resulting resources is scaled down to its original decimals.

```
function _convertToShares(address vault, uint256 assets, uint256 price,
    Math.Rounding rounding)
// ...
{
    uint8 decimals = IERC20Metadata(vault).decimals();
    uint256 sharesScaled = _scaleUp(assets, decimals).mulDiv(10 **
        _sharePriceDecimals(), price, rounding);
    shares = _scaleDown(sharesScaled, decimals);
}

function _convertToAssets(address vault, uint256 shares, uint256 price,
    Math.Rounding rounding)
// ...
{
    uint8 decimals = IERC20Metadata(vault).decimals();
    uint256 assetsScaled = _scaleUp(shares, decimals).mulDiv(price, 10 **
        _sharePriceDecimals(), rounding);
    assets = _scaleDown(assetsScaled, decimals);
}
```

The last scaling down actually removes rounding-up if the `rounding` was requested as `Ceil`.

There are two instances where rounding is done up, in `deposit` and in `redeem`. In both cases the rounding protects against odd corner-cases.

An example of abuse that is possible because of the faulty rounding is unauthorized share burning that can lead to vault share depletion. The issue is described further.

A controller can initiate a redemption by calling `AccountableAsyncVault::requestRedeem`, which transfers their shares to the vault and creates a pending redeem request.

The loan operator then fulfills this request in `AccountableLoan::fulfillRedeemRequest`, setting `state.maxWithdraw` to the maximum amount of **assets** the controller can claim in exchange for their shares.

The controller can then call `redeem`, specifying the number of shares they want to convert into assets. An issue arises when the controller redeems an amount of shares that, when converted, is rounded down to 0 assets. In this case:

- The protocol burns the specified shares from the vault.
- `state.maxWithdraw` remains unchanged since no assets are withdrawn.

This creates an attack vector where an attacker can repeatedly burn more shares than he initially deposited. This results in burning shares belonging to other users that have them currently pending.

When legitimate users attempt to claim their redeem or deposit requests, their requests may fail due to the vault lacking enough shares to burn.

Recommendation

Both in the `_convertToAssets` and `_convertToShares` functions, completely remove the `_scaleUp` and `_scaleDown` logic as they have no precision benefit whatsoever and introduce the mentioned rounding issue.

Example implementation:

```
function _convertToShares(address vault, uint256 assets, uint256 price,
    Math.Rounding rounding)
// ...
{
    shares = assets.mulDiv(10 ** _sharePriceDecimals(), price, rounding);
}

function _convertToAssets(address vault, uint256 shares, uint256 price,
    Math.Rounding rounding)
// ...
{
    assets = shares.mulDiv(price, 10 ** _sharePriceDecimals(), rounding);
}
```

There is no need for scaling as the `AccountableVault` constructor sets the share decimals to that of the asset decimals.

Resolution: Resolved, the recommended fix was implemented in [#640f2e1](#).

[H-04] Unauthorized redeem requests allow forced redemptions and grieving

Severity: *High risk* (Resolved) [PoC]

Context: *AccountableAsyncVault.sol:147*

Description

To initiate an asynchronous redeem, the `AccountableAsyncVault::requestRedeem` function must be called.

Due to a missing check, the `requestRedeem` can be called by anyone on behalf of any share owner. This allows an attacker to:

- Forcefully submit redeem requests for any user.
- Grief users by blocking their attempts to submit a redeem request.

Consider the following scenario:

- `alice` submits a request to redeem all the `X` shares she owns.
- `bob` front-runs her transaction with a redeem request for 1 wei of `alice`'s shares.
- `bob`'s request is processed first, transferring 1 wei of shares from `alice` to the vault.
- `alice`'s transaction now reverts because she only has $X - 1$ shares left, which is insufficient for her original request.

Also, as anyone can fully redeem any other user, the entire protocol can be basically blocked.

Recommendation

Restrict access to the `requestRedeem` function by ensuring only authorized operators can call it on behalf of a user by adding the `_checkOperator(owner)` check.

Resolution: Resolved, the recommended fix was implemented in [#640f2e1](#).

3.3 Medium Severity Findings

[M-01] Accountable vaults are not compliant with ERC-4626, ERC-7540 and ERC-7575

Severity: *Medium risk* (Partially Resolved)

Context: [AccountableAsyncVault.sol](#) [AccountableVault.sol](#)

Description

ERC (Ethereum Request for Comment) compliance is an important aspect to any protocol that wishes to integrate and be integrated with by the broad blockchain ecosystem. Non-compliance with the [ERCs](#) can lead to slow market adoption and, at worst, issues with integrating third parties.

The `AccountableAsyncVault` contract and its parent `AccountableVault`, was designed to be [ERC-7540](#) compliant, which implies extending the [ERC-4626](#) standard and several other adjacent standards, such as [ERC-7575](#).

The existing vault implementation, coupled with the existing loan manager, displays incompatibility and non-compliance with the aforementioned standards due to several issues.

ERC-4626 compliance issues

- the `maxRedeem` function specifically *MUST NOT revert* but `AccountableAsyncVault::maxRedeem` will revert before the first redeem request is fulfilled for the owner, because `state.redeemPrice` is only set during the first fulfill call and `maxRedeem` will use it as division denominator, meaning it will revert with a division by zero panic error
- `maxDeposit`, `maxMint`, `maxRedeem` and `maxWithdraw` do not take into account whether the loan implementation contract is paused or if the loan is operational, as opposed to the EIP requirements

MUST factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily) it MUST return 0.

ERC-7540 compliance issues

- `Deposit` event, emitted in the `AccountableAsyncVault::deposit` function, incorrectly uses the receiver as the first parameter, as opposed to the standard:

When the `Deposit` event is emitted, the first parameter MUST be the controller, and the second parameter MUST be the receiver.

- ERC-165 support is not fulfilled as the vaults don't implement the `ERC7575` standard, thus missing the `0x2f0a18c5` interface id, and the `0xe3bc4e65` id, corresponding to the `IControllerOperator` interface, must be separately supported, as mentioned in:

All asynchronous Vaults MUST return the constant value true if either 0xe3bc4e65 (representing the operator methods that all ERC-7540 Vaults implement) or 0x2f0a18c5 (representing the ERC-7575 interface) is passed through the interfaceID argument.

ERC-7575 compliance issues

The ERC7540 standard specifically requires ERC-7575 to be supported, which it is not

Smart contracts implementing this Vault standard MUST implement the ERC-7575 standard (in particular the share method).

Recommendation

- in `AccountableLoan::maxRedeem` check if price is 0 and return 0 instead of calculating share price
- in `maxDeposit`, `maxMint`, `maxRedeem` and `maxWithdraw` from the `AccountableLoan` contract, check if the contract is paused and return 0. Interface changes will probably be necessary as the pausing logic is in the `AccountableFixedTerm` contract
- in `AccountableAsyncVault::deposit`, emit the `Deposit` event with `controller` as the first parameter and `receiver` as the second
- in the `AccountableAsyncVault::supportsInterface` function, also check for `interfaceId == type(IControllerOperator).interfaceId` and for the ERC7575, after implementing it
- implement the ERC-7575 standard in the `AccountableVault` contract.

Resolution: Partially resolved by the team in [#640f2e1](#).

Accountable: implemented all recommendations without ERC7575, because that's not backwards compatible with ERC4626, and we care about that right now. considering support for it in future versions.

[M-02] Frontrunning fulfillment transactions leads to vault depletion

Severity: *Medium risk* (Resolved) [\[PoC\]](#)

Context: [AccountableLoan.sol:242,262](#)

Description When a user submits a deposit request, it enters a pending state until the loan operator fulfills it. During this period, the user can also request to cancel their pending deposit, and the loan operator is expected to process the cancellation for the full deposit amount.

However, in the `AccountableLoan` contract, the `fulfillDepositRequest` function does not check whether the deposit being fulfilled is pending cancellation. This creates an opportunity for an attacker to frontrun the loan operator's transaction with a deposit cancellation request.

As a result:

- The deposit cancellation request enables the attacker to get their original assets back

-
- The fulfill deposit request enables the attacker to get the corresponding shares for their assets
 - The attacker can then redeem their shares in exchange for assets that don't belong to them

This exploit effectively allows an attacker to drain the vault.

The same issue exists in the redemption process, where an attacker can frontrun `fulfillRedeemRequest` with a cancel request to double their claimable assets.

Recommendation

In `fulfillDepositRequest` and `fulfillRedeemRequest`, add a validation check to ensure the controller does not have a pending deposit cancellation request or a pending redeem cancellation request, respectively.

Resolution: Resolved, the recommended fix was implemented in [#640f2e1](#).

[M-03] Restrictive whitelisted-lender permission model introduces economical constraint

Severity: *Medium risk* (Resolved)

Context: [AccountableLoan.sol](#)

Description

When a redeem request is submitted, the `AccountableFixedTerm::_authOwner` function ensures either the asset owner or their operator is a whitelisted lender. Similarly, canceling a redeem request requires the controller to be a whitelisted lender. Claiming a redeem request requires the controller or the assets' receiver to be a whitelisted lender.

By using this design, severe limitations and economical constraints are added to the vault share tokens as the tokens can be transferred and sold to any arbitrary user, but if the new owner of the share is not a whitelisted lender, he cannot withdraw his deposit (if available and not used by the borrower).

The overall effect is that no extra DeFi layer can be built on top the shares, as they have economical value only to whitelisted lenders.

This design and approach also applies to deposits but in the case of deposits it is valid, as only trusted and whitelisted lenders (meaning depositors) are approved by the protocol.

The current permissioning model also introduces an issue when an operator submits a deposit request on behalf of a whitelisted lender without being a whitelisted lender themselves. The operator can initiate the deposit but is restricted from canceling or claiming it for themselves. The deposit can only be claimed if the receiver is an authorized lender.

A further problematic situation appears when a whitelisted lender has funds in the vaults but his whitelisted status is removed. At that point, he is unable to withdraw his tokens.

Recommendation

In the current implementation of `AccountableLoan`, the `_authOwner` permission check is used on all relevant callbacks. Specifically `_authOwner` is used in the: `requestRedeem`, `redeem`,

`withdraw`, `cancelRedeemRequest`, `claimCancelRedeemRequest`, `requestDeposit`, `deposit`, `mint`, `cancelDepositRequest` and `claimCancelDepositRequest` functions.

Having one single validation logic for 10 different actions severely limits permission granularity management, as such, create a different authentication function check for each action or group of actions.

For example one `_authRedeemParticipants` and `_authDepositParticipants` to validate redeems/withdrawals and deposits/mints (claims) where semantically there is a receiver and a controller checked, not an owner and a controller, [as the current implementation considers](#) by reusing `_authOwner`.

With the above new authentication functions, ensure that redeem related operations do not depend on the participants being whitelisted lenders.

Also, consider if allowing the mentioned case of non-whitelisted controller being allowed to initiate a request but not to claim it is acceptable and or not. If not desired, modify the granularity logic accordingly.

Resolution: Resolved, the recommended fix was implemented in [#a3976b2](#).

3.4 Low Severity Findings

[L-01] Protocol does not support asset decimals greater than share price decimals

Severity: *Low risk* (Resolved)

Context: [AccountableLoan.sol:358,364](#)

Description

The AccountableLoan contract relies on the `_scaleUp` and `_scaleDown` helper functions to convert between assets and shares.

These functions adjust the precision of a given value based on the decimal difference

`_sharePriceDecimals() - decimals`, presuming the asset decimal is always less or equal to the shares decimals. However, if `decimals > _sharePriceDecimals()`, this calculation will underflow and cause a revert.

As a result, users can submit deposit requests, but their fulfillment will always fail due to the revert. The only option users have is to cancel the deposit request and reclaim their assets once the cancellation is processed.

Recommendation

Both in the `_convertToAssets` and `_convertToShares` functions, completely remove the `_scaleUp` and `_scaleDown` logic as they have no precision benefit.

Resolution: Resolved. A fix was implemented in [#640f2e1](#).

Accountable: removed scaling in conversions, didn't remove the scale functions.

[L-02] Incorrectly fulfilling requests breaks accounting

Severity: *Low risk* (Resolved)

Context: [AccountableLoan.sol:256,294,278,312](#)

Description

When a user submits a request, it enters a pending state. The user can then request to cancel their pending request. In both cases the loan operator is expected to fulfill either the request or cancellation.

However, in both fulfillment cases and for each operation type (deposit and redeem/withdraw) there are no checks if the loan operator is not actually fulfilling and committing more assets to the underlying request.

If, by mistake or in extreme cases, such as a blockchain reorg, the loan operator calls the fulfillment functions with incorrect values, the extra allocated resources will lead to losses for some users and gains for another.

The underlying issue is present in each pair of functions for each operation. For redeems, in the `fulfillRedeemRequest` and `fulfillCancelRedeemRequest` functions, the `state.pendingDepositRequest` amount is not checked to be enough to cover the requested shares and simply trimmed, allowing excess shares/assets to be allocated.

```
state.pendingRedeemRequest = state.pendingRedeemRequest > shares ?  
    state.pendingRedeemRequest - shares : 0;
```

Likewise, with the deposit operation, in the `fulfillDepositRequest` and `fulfillCancelDepositRequest` functions, the `state.pendingDepositRequest` amount is not checked to cover requested shares and is trimmed, leading to the same possible over-allocation.

```
state.pendingDepositRequest = state.pendingDepositRequest > assets ?  
    state.pendingDepositRequest - assets : 0;
```

An example of how this issue might appear on blockchains with reorgs. Consider the following sequence of transactions:

- Tx #1: bob requests to deposit X assets
- Tx #2: bob requests to deposit Y assets
- Tx #3: bob requests to cancel his pending deposits
- Tx #4: The loan operator fulfills bob's cancellation request for X + Y assets

Chain Reorg Scenario #1 (Tx order 1 → 3 → 2 → 4):

- Tx #1 succeeds
- Tx #3 succeeds
- Tx #2 reverts with `CancelDepositRequestPending`
- Tx #4 makes X + Y assets available for bob to claim ⇒ bob retains the Y assets that were never successfully deposited while also having X + Y claimable assets from the cancel request. bob effectively extracted Y assets from the vault

Chain Reorg Scenario #2 (Tx order 1 → 3 → 4 → 2):

- Tx #1 succeeds
- Tx #3 succeeds
- Tx #4 makes X + Y assets available for Bob to claim
- Tx #2 sets bob's pending deposit amount to Y ⇒ bob now has a pending deposit of Y that will be fulfilled by the loan operator while also having X + Y claimable assets from the cancel request. bob effectively extracted Y assets from the vault

The reorg issue subcase will manifest itself naturally in blockchains with a high reorg rate, such as Polygon POS Chain, but for the in-scope mentioned chains, it is high unlikely for it to manifest.

Recommendation

For deposits, in the `fulfillDepositRequest` and `fulfillCancelDepositRequest` functions, ensure that the user has a sufficient pending deposit balance before fulfilling the operation. This can be done by reverting if `state.pendingDepositRequest < assets`. Also, the ternary if condition that trims excess assets should be removed and simply deduct from `state.pendingDepositRequest` the assets amounts.

Likewise for redeems, in the `fulfillRedeemRequest` and `fulfillCancelRedeemRequest` functions, ensure that the user has a sufficient pending redeem balance before fulfilling the operation. Doable by reverting if `state.pendingRedeemRequest < shares`. And also in this case, the ternary if condition that trims excess shares be removed and replaced with a simple deduction of shares from `state.pendingRedeemRequest`.

Resolution: Resolved, the recommended fix was implemented in [#640f2e1](#).

[L-03] Possible event spamming due to missing 0 input amount checks

Severity: *Low risk* (Resolved)

Context: [AccountableLoan.sol:332-365](#)

Description

The current implementation in `AccountableLoan` allows several functions of the `AccountableAsyncVault` to be called with a 0 amount input. The user facing functions from `AccountableAsyncVault` are:

- `deposit`
- `mint`
- `redeem`
- `withdraw`

Calling these functions with a 0 input does not result in any state changes besides event emission, but by abusing this oversight, off-chain event spamming can be done. Also, calling these functions randomly should not be possible in a state-like system.

Recommendation

In the `AccountableAsyncVault` contract, ensure that all user facing functions do not allow a 0 input.

Resolution: Resolved, the recommended fix was implemented in [#cddf102](#).

[L-04] Vault claim events can be spammed

Severity: *Low risk* (Resolved)

Context: [AccountableAsyncVault.sol:218,141](#)

Description

Users can cancel their pending requests and afterwards, after fulfillment, claim the resources back.

Claiming shares from canceled redeem requests is done via the `claimCancelRedeemRequest` function and, similar, assets from canceled deposits are claimed via the `claimCancelDepositRequest` function from the `AccountableAsyncVault` contract.

These function, incorrectly can be called when users have no pending asset or shares to claim. While no on-chain storage changes are made, both functions emit events that may confuse third party integrators and off-chain monitoring tools.

Recommendation

In the `AccountableAsyncVault` contract, check that the resulted amounts from the `claimCancelRedeemRequest` and `claimCancelDepositRequest` functions, from the `AccountableLoan` contract, are greater than 0, otherwise revert.

Resolution: Resolved, the recommended fix was implemented in [#cddf102](#).

3.5 Informational Findings

[I-01] Transfer whitelists can't be added

Severity: *Informational* (Resolved)

Context: *AccountableAsyncVault.sol*

Description

In the `AccountableAsyncVault` vault, there is a `transferWhitelist` mapping used to allow transferring of tokens even if the transfers are disabled.

Besides the loan manager, which is added at deployment, there is no other mechanism to add to this mapping.

Recommendation

Add a gated `addTransferWhitelist` function equivalent in the `AccountableAsyncVault` vault.

Resolution: Resolved, the recommended fix was implemented in [#640f2e1](#).

[I-02] Add indexing for `IAccountableLoan` events

Severity: *Informational* (Resolved)

Context: *IAccountableLoan.sol:346-377*

Description

In the `IAccountableLoan` interface, none of the existing events use `indexed` for their parameters.

This reduces third party visibility and monitoring capability while not providing any benefit for the protocol.

Recommendation

Add `indexed` to all relevant parameters of the `IAccountableLoan` interface events.

Resolution: Resolved, the recommended fix was implemented in [#640f2e1](#).

[I-03] Rounding discrepancy between fulfilling and claiming can cause inconveniences

Severity: *Informational* (Acknowledged) *[PoC]*

Context: *AccountableLoan.sol:79-83,177-182*

Description

In the `AccountableLoan` contract, when a deposit request is fulfilled by the loan operator via `fulfillDepositRequest`, the user's `state.maxMint` is increased by the rounded down number of shares. This ensures the rounding favors the protocol.

However, when a user claims their deposit via the `deposit` function, the provided asset amount is compared against `state.maxMint` converted back into assets. Because `state.maxMint` was initially rounded down, converting it back may yield a lower asset amount than what the user originally deposited, leading to a revert with `ExceedsMaxDeposit`.

Similarly, when converting the asset amount to shares, the rounded up share amount might exceed `state.maxMint`, causing a revert with `ExceedsDepositLimit`.

As a result, users may be unable to claim their deposit even when providing the same asset amount as in their original request.

Note that the same issues occur in the `redeem` function.

Recommendation

The above issue appears only if user presumes that the requested amount is equal to the claimed amount. The indicated rounding down/up behavior protects the protocol from various other issues and abuses.

Since users may simply use the `maxMint` and `maxRedeem` values themselves when initiation full deposit/redeem, instead of the requested amount, the issue will not even manifest.

Rounding is a crucial aspect of any borrowing and lending protocol, and as such, the current behavior, which favors the protocol, is beneficial.

Thus, *acknowledge this issue* and document that users should use the `maxMint/maxDeposit` and `maxRedeem/maxWithdraw` functions to get their full amounts out, not the requested amount as rounding errors might appear and cause a few WEI loss.

Resolution: Acknowledged by the team.

[I-04] Missing event emission for important actions

Severity: *Informational* (Resolved)

Context: *[AccountableAsyncVault.sol:322-325](#) [AccountableFixedTerm.sol](#)*

Description

Throughout the codebase there are instances where important setter functions, or actions, are executed but no event is emitted.

- in `AccountableAsyncVault`, in the `setSharesTransferable` function, there is no emitted event
- in `AccountableFixedTerm`, after the following functions are executed, an event should be emitted

-
- setKycRegistry
 - setInvestmentManager
 - setSafetyModule
 - setVault
 - setDepositGracePeriod
 - setLoanStartGracePeriod
 - setLateInterestGracePeriod

Lack of events in these instances increases the difficulty in off-chain protocol monitoring by third parties.

Recommendation

Add events in each of the mentioned cases.

Resolution: Resolved, the recommended fix was implemented in [#a3976b2](#).

[I-05] Soulbound vault shares can be transferred by abusing the redeemRequest and cancelRedeem mechanism

Severity: *Informational* (Acknowledged)

Context: *AccountableAsyncVault.sol*

Description

The AccountableVault contract has a transferableShares function, set to true in the current implementation, that dictates if users are allowed to transfer shares or not.

This mechanism can be subverted and shares transfer by abusing the AccountableAsyncVault vault redeemRequest and cancelRedeem functions.

To transfer from bob to alice, bob can:

- initiate a requestRedeem with himself as controller and also calls cancelRedeemRequest in the same block to ensure funds are not borrowed
- loan operator fulfills the cancel redeem request at one point
- bob now calls redeem with the receiver set to alice, thus finalizing a transfer

Since the redeem function withdraws shares while bypassing any transfer constraint, it effectively acts as a proxy and allows transfers.

Recommendation

To completely eliminate this issue an extra internal tracking mechanism for each request must be implemented, possibly relying on a REQUEST_ID different from 0, and track the owner of each request, so that in the redeem and withdraw functions, receiver to be limited to the initial owner. This implies extreme overhead and may not be feasible without further limitations, such as not allowing any operators to be set, as any operator can initiate a cancelRedeemRequest request and a redeem request.

A compromise can be achieved by modifying the redeem and withdraw mechanism so that, if the shares are soulbound, only allow the receiver to be the controller. Share owners will not use this mechanism to transfer to third party addresses, as it exposes their other assets in the system. Although this still allows transfers between approved controllers and transfers of complete positions to 3rd parties, it only limits partial share transfers.

As the current implementation does not use this feature, the issue can be acknowledged, but any future extension must take the above into consideration.

Resolution: Acknowledged by the team.

[I-06] Missing input sanitization on important setters

Severity: *Informational* (Resolved)

Context: *AccountableFixedTerm.sol:119-122,131-167*

Description

Throughout the codebase there are two instances where setters are called with unvalidated input, which may result in protocol issues, such as blocking functionality until the values are changed again.

In `AccountableAsyncVault::constructor` the `loanManager_` address is not checked to not be the 0 address

In `AccountableFixedTerm`:

- in `initialize` all addresses 4 address type parameters are not checked for the 0 address
- in `initLoan` and in `updateLoan`, there is no sanity check on the loan terms. Checks to add:
 - that the `interestInterval` is less than the `duration`
 - that `interestRate` is below `BASIS_POINTS`

Recommendation

Add validations on the inputs in the mentioned instances.

Resolution: Resolved, the recommended fix was implemented in [#a3976b2](#).

[I-07] `_totalPendingRedeemAssets` is not used

Severity: *Informational* (Acknowledged)

Context: Global

Description

In the `AccountableLoan` contract, when a user initiates a withdraw or redeem, the total pending assets to redeem are increased. Subsequently, when actually finalizing the request, they are subtracted.

While the amounts are tracked in the `_totalPendingRedeemAssets` storage variable they are never used outside internal tracking. There is also no equivalent total pending deposit assets or total pending canceled assets.

Recommendation

Create a getter for the `_totalPendingRedeemAssets` value. Also consider if the other, overall amounts, are of interest to be tracked.

Resolution: Acknowledged by the team.

: not being used is fine - this is intended, some strategies will need total assets requested. more interested if the sum is in sync with the shares outstanding for individual redemptions.

[I-08] Remove debug remnants from vaults

Severity: *Informational* (Resolved)

Context: *AccountableVault.sol:4,12 AccountableLoan.sol:11,4*

Description

Throughout the codebase there are debug remnants from the development phase.

- in `AccountableVault` the `IERC165.sol` and `console` imports are unused
- in `AccountableLoan` the `console` import is unused
- in `AccountableAsyncVault`, the `TODO: think this is not needed` can be removed along with the line itself, as the current implementation does not require it

Recommendation

Remove the indicated debug remnants.

Resolution: Resolved, the recommended fix was implemented in [#a3976b2](#).

[I-09] Add security contact to contracts

Severity: *Informational* (Resolved)

Context: Global

Description

In case a whitehat identifies an issue with the on-chain contracts of the protocol, to more easily be able to contact the team, have a security contact added to the contracts.

Recommendation

Add the email address for the team security contact either as a plain comment or as a `custom:security-contact natspec` tag to the contracts.

Resolution: Resolved, the recommended fix was implemented in [#640f2e1](#).

4 System design notes

4.1 ERC-7540 integration

The ERC-7540 standard allows for async logic on redeems, deposits or both (not on none). For the current logic, decide if a deposit async logic is required.

To deposit into the system, the lender whitelist is already required (except when the permissioning level is set to none). Having an extra requirement for an async fulfill on deposits may be redundant. Plus, if a tranche logic is added, further gatekeeping can be implemented there.

4.2 Loan managers

While the current `AccountableFixedTerm` contract is used as a draft for a fixed term loan manager, some observations and notes can be shared in regard to any future implementation.

1. Clearly define the loan amount

Careful consideration must be given to what is considered the actual borrowed amount. If the off-chain legal binding contract specifies one amount, the on-chain implementation should reflect that, or vice-versa to ensure no ambiguity.

Ambiguity can appear if on-chain borrower borrows (releases) amount X but the legal contract indicates a different amount. Share pricing, which most likely will account for this amount in one way or another, must also be priced accordingly.

2. Is borrower allowed to re-borrow

A loan manager may allow borrower to early repay and, again, take a loan. Discrepancies may appear if the legal agreement indicates a maximum amount (equivalent to `maxLoanAmount`) as allowed or a maximum volume?

e.g. borrowing 100,000 USD, repaying 50,000 USD and then borrowing again 25,000 USD. Is this a \$125,000 or a \$100,000 loan?

A possible approach may be to define in the loan manager the “want” amount for a borrower and only allow depositors to deposit up to that amount.

3. Payment interval odd cases

The current implementation sets the first required payment interval since the loan start, but subsequent interest payments will set the new start time to that moment. This is one way of designing the system. Another design would be to consider each payment (plus late fees) only covering a set new interval (e.g. old payment date + duration), instead of (current date + duration).

4. Lender expectation

Depending on intention, the system may be design to ensure that all depositors (lenders) receive interest. But borrowers may not actually use the lender deposited amount.

In that case, is the lender still entitled to interest? These must be clearly outlined both formally and in code. The current system presume so, thus some correlations should be done on either deposited amount or allowed to borrow amounts to please all market participants.

Also, if borrower repays in advance, should interest also be deducted for the early repay and accounted to lenders? Currently interest is taken even if an early repay is done, worst case at loan finalization.

5. Interest calculation

Claiming interest is not well-defined in the existing implementation as a separate action. Some observations into adding such a separate feature:

- all lenders must have their entry point in the same contract, thus in the ERC-7540 vault
- a `claimInterest` interest function may be added, but it should calculate the intended interest amount (in share equivalent) and initiate a `redeemRequest`, to maintain protocol cohesiveness

The actual interest amount calculation may differ. The most used implementation, and mentioned by the team, is to consider it a proportion of the shares owned relative to total existing shares. Several points must be considered in this case:

- since shares are burned on redeem only, are pending-to-be-claimed shares taken into consideration?
- special attention must be given to the interest distribution (more so if deposits are ever made atomically) to ensure no flash-loans can be taken and timed with an interest distribution so that to extract value without incurring any risk, as the lender
- consider should there be a required minim shares holding time before gaining interest? Valid implementations may exist without adding such as constraint by ensuring distribution is made fairly

Using owned percent of total shares is a viable implementation, with the added points to be accounted for.

6. Reusable loan managers

Another design decision is regarding usability. Ideally, after a loan has been repaid, the same loan manager and vault can be reused with new loans. Current loan managers implementation does not support this, but it may be fairly easy extended to support it.

7. Escrow funds

Depending on:

- how interest calculation is done
- if early repays are calculated with interest
- (if added) how complex the tranche/gatekeeping logic is

It may be feasible to move all funds to an escrow and not in the vaults themselves. Although, at this point, this design change should only be considered if such a point is reached, or if smart contract code size reaches critical values.

8. Defaulting behavior

When a loan defaults, pending deposits should not be allowed and existing shares/redeems should enter a different mechanism.

A simple, but somewhat controversial mechanism, would be to allow the loan manager to distribute existing funds given a list of addresses and amounts arbitrarily. That way the decision of how remaining funds are split is left up to off-chain, legal bindings.

The existence of a safety module is also relevant to the default behavior.

The current safety module may be modified to use something similar to AAVE's [Safety-module](#), with extra interest incentives for stakes, but this will require more implementation to integrate.

The existing `ISafetyModule` interface is also lacking an opt-in type of function, implying that market participants would not voluntarily participate. Depending on what rewards do safety module participants receive, this may be an incorrect point of view. Add interface functions such as [stake/redeem](#) or an opt-in/opt-out equivalent.

Note:

- removing a safety module for a started loan will effectively change lenders risk parameters and either should not be allowed or done with the utmost consideration as to avoid an equivalent bank-run due to lender risk increase
- it may be economically valid to have a separate SvaaS (Safety Vault as a Service) project which can handle multiple scenarios

4.3 Tranche tokens

If the team decides to support [tranche](#) type tokens, there are two approaches that that can be taken:

- the `AccountableVault` contract can be extended to include the restriction logic, maybe including a tranche manager like component
- the vault share themselves may be created as a different contract and have the logic included in that specific contract (ERC-7540 supports this option)

The second method is used by the [Centrifuge](#) with various degrees of success.

An EVM standard for tranche tokens that is universally accepted does not exist, however, the closest is [ERC-1404](#) (<https://erc1404.org>) which is an extension of ERC-20.

It is important to note that any blacklisting and gatekeeping may result in lender funds being blocked in the protocol. This may be intended, and in that circumstance, consider implementing mechanism to forcefully remove blocked lender funds or integrate them correctly in the contract accounting.

4.4 General code advice

Eliminate subtractions when possible

Always prefer additions before subtractions to eliminate any possible underflow situation.

Example:

```
if (block.timestamp - _loan.startTime > _loan.duration) revert LoanNotMatured();
```

is better written as:

```
if (block.timestamp > _loan.startTime + _loan.duration) revert LoanNotMatured();
```

Vault upgradability

Consider if adding an upgradable pattern to the vaults (not loan managers) is valid. A [beacon proxy pattern](#) may be used in this case which ensure that any deployed vault is instantly upgraded if needed. While this does add centralization, it allows for flexible and quick updates. The team should consider if this is relevant for them.

Fulfillment validations

Request fulfillment depends on off-chain validation logic which then can call the on-chain functions. Do not rely solely on off-chain validation and implement, as much as possible, sanity checks and validations in all fulfil-type functions to ensure, by mistaken, no odd case is introduced. Having a trusted role that calls this should not be the only relied-upon safety feature.

5 Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts the consultant to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. The consultant’s position is that each company and individual are responsible for their own due diligence and continuous security. The consultant’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology that is analyzed.

The assessment services provided by the consultant is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. Furthermore, because a single assessment can never be considered comprehensive, multiple independent assessments paired with a bug bounty program are always recommend.

For each finding, the consultant provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved, but they may not be tested or functional code. These recommendations are not exhaustive, and the clients are encouraged to consider them as a starting point for further discussion.

The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties. Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, the consultant does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

The consultant retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. The consultant is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. The consultant is furthermore allowed to claim bug bounties from third-parties while doing so.